
OPENCV DESDE PYTHON:

INTRODUCCIÓN A LA CALIBRACIÓN DE CÁMARAS

INTRODUCCIÓN

OpenCV utiliza el método de calibración de Zhang que utiliza al menos tres imágenes capturadas desde diferentes puntos de vista de un patrón planar. Los patrones planares tienen la ventaja de que pueden imprimirse con la precisión de una impresora. En los proyectos de esta asignatura, se deberán calibrar las cámaras utilizando este método para obtener modelos precisos de las mismas. Los modelos calibrados de las cámaras servirán para proyectar los objetos virtuales sobre las imágenes capturadas y generar realidad aumentada de manera efectiva, así como también para localizar los objetos reales en el entorno, lo que permitirá una interacción más precisa y realista entre el mundo virtual y el mundo real. Esto abre un abanico de posibilidades para aplicaciones innovadoras en campos como la robótica, la medicina, la arquitectura, el entretenimiento y muchos más, donde la realidad aumentada y la visión 3D desempeñan un papel fundamental.

OBJETIVOS

El objetivo principal de esta sesión de prácticas es aprender a calibrar una cámara y utilizar el modelo calibrado para proyectar al menos un punto virtual sobre las imágenes capturadas por la cámara utilizando OpenCV desde Python.

MATERIALES

Para desarrollar esta práctica, descargue los siguientes materiales a una carpeta de trabajo local (en adelante, "[working_folder_path](#)") en el ordenador donde vaya a ejecutar el cuaderno (*.ipynb) o el fichero Python (*.py).

- PoliformaT -> Recursos/2-Software-&-cuadernos/50-Jupyter lab Tutorial – Camera model
- PoliformaT -> Recursos/2-Software-&-cuadernos/51-Jupyter lab Tutorial – Camera calibration
- PoliformaT -> Recursos/2-Software-&-cuadernos/52-PyCharm demo - Camera calibration

Importante >> En la carpeta "52-PyCharm demo - Camera calibration/patterns" encontrareis tres ejemplos de patrones de calibración que podéis imprimir para calibrar vuestra cámara.

Importante >> No olvide hacer una copia de seguridad del código implementado (por ejemplo, en su carpeta W:) al finalizar la práctica.

1.- ARRANQUE CON Jupyter lab

Si nunca has utilizado PyCharm, es mejor empezar con Jupyter lab. Ejecuta el programa Jupyter lab desde un terminal "Anaconda Prompt (miniconda 3)", tras activar el "environment" adecuado.

```
(base) C:\windows\system32> conda activate open3d_env  
(open3d_env) C:\Users\username> jupyter lab --notebook-dir "working\_folder\_path"
```

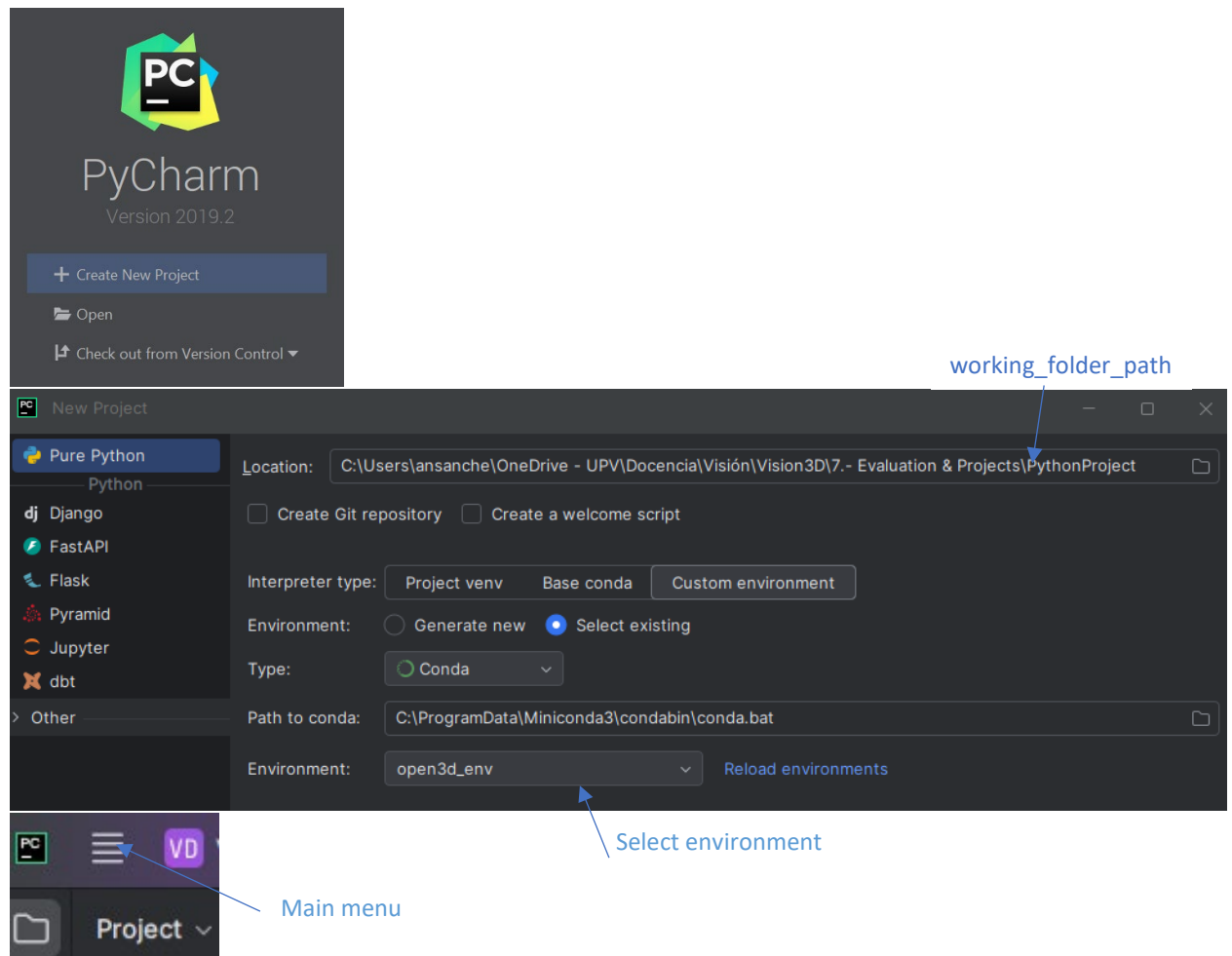
Después de arrancar Jupyter lab, escribe en una célula de código el comando: `"import cv2"`

Mediante este módulo, Python encontrará todos los métodos de Python que sirven de interfaz entre la librería OpenCV de C++ y Python.

Utilidad >> Para habilitar el autocompletado de código en JupyterLab, presione la tecla Tab mientras escribe el código.

1.2.- ARRANQUE CON PyCharm

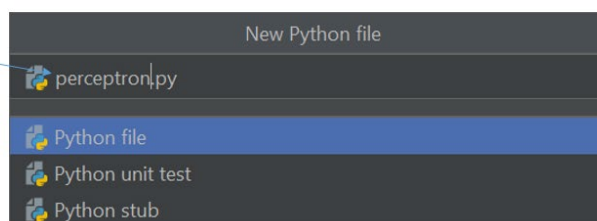
Si prefieres comenzar a trabajar con PyCharm, deberás crear un proyecto y seleccionar el “environment” adecuado desde el desplegable “Environment”.



Para crear un nuevo fichero Python (*.py) simplemente selecciona la siguiente opción del “Menu principal”:

- Main menú -> File -> New -> Python File

New file name



2.- DETECCIÓN DE PUNTOS

Analizar la siguiente función de detección de esquinas en un patrón tipo tablero de ajedrez implementada en la demo "52_Camera_Calibration_demo.py":

```
def getCalibrationPoints(fname):

    # Defining the dimensions of checkerboard
    patternSize = (6,9)

    # Defining the world coordinates for 3D points
    objp = np.zeros((1, patternSize[0] * patternSize[1], 3), np.float32)
    objp[0,:,2] = np.mgrid[0:patternSize[0], 0:patternSize[1]].T.reshape(-1, 2)

    # Find the chess board corners
    img = cv2.imread(fname)
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    imgSize = gray.shape[::-1]

    # If desired number of corners are found in the image then ret = true
    ret, corners = cv2.findChessboardCorners(gray, patternSize, None)

    if ret == True:

        # Draw and display the corners
        img = cv2.drawChessboardCorners(img, patternSize, corners, ret)

        cv2.imshow('img', img)
        cv2.waitKey(0)
        cv2.destroyAllWindows()

    return ret, corners, objp, imgSize
```

Esta función se puede modificar para detectar los centroides de los círculos en un patrón de círculos:

```
o ret, centres = cv2.findCirclesGrid(gray, ...
    patternSize, None, flags=cv2.CALIB_CB_SYMMETRIC_GRID)
```

Tened en cuenta si es simétrico (opción por defecto) o si es asimétrico y configurarlo con los flags 'cv2.CALIB_CB_ASYMMETRIC_GRID'. En el caso del asimétrico también es necesario modificar los parámetros por configuración del detector para aumentar el tamaño del área del círculo con el siguiente argumento del cv.findCirclesGrid:

```
o params = cv2.SimpleBlobDetector_Params()
  params.maxArea = 10000
  detector = cv2.SimpleBlobDetector_create(params)
```

Además, se dispone de la función que permite dibujar los puntos detectados sobre las imágenes.

```
o img = cv2.drawChessboardCorners(img, patternSize, centres, ret)
```

3.- CALIBRACIÓN

Analizar el siguiente script de calibración:

calibrateCamera

```
opts = []
ipts = []

# Extracting path of individual image stored in a given directory
Images = glob.glob('./CalibrationImages/*.jpg')
for fname in images:
    ret, corners, objp, imgSize = getCalibrationPoints(fname)

    if ret == True:
        opts.append(objp)
        ipts.append(corners)

ret, mtx, distCoeffs, rvecs, tvecs = cv2.calibrateCamera(opts, ipts, imgSize, None, None)
print("\nCamera matrix : ")
print(mtx)
print("\ndistCoeffs : ")
print(distCoeffs)
print("\nrvecs : ")
print(rvecs)
print("\ntvecs : ")
print(tvecs)
```

4.-TAREA

A partir de los materiales y ejemplos anteriores deberéis integrar en vuestro proyecto:

- un método que capture imágenes de la cámara, capture X imágenes, muestre los puntos del patrón (esquinas de los cuadrados o centros de los círculos) para que el usuario pueda confirmar su adquisición y cuando llegue a X imágenes calcule la calibración. Además, podríais:
 - recibir argumentos de entrada el tipo (chessboard o circleGrid) y los parámetros del tablero de calibración (número de filas x columnas),
 - guardar los resultados de calibración en un fichero,
- otro método que utilice el modelo de la cámara para proyectar un punto virtual sobre las imágenes capturadas por la cámara.